

Project Structure

A guided tour of the Automotive Academy codebase

"Show me your folder structure and I'll tell you how you think." (every tech lead ever)

→ *core/* has no Android UI: it's the brain

→ *ui/* only renders state

→ one seam: `VehicleDataSource`

A Head First-style learning guide that goes with the **Automotive Academy** sample app (Android Automotive OS, Kotlin, Jetpack Compose). Generated September 2026.

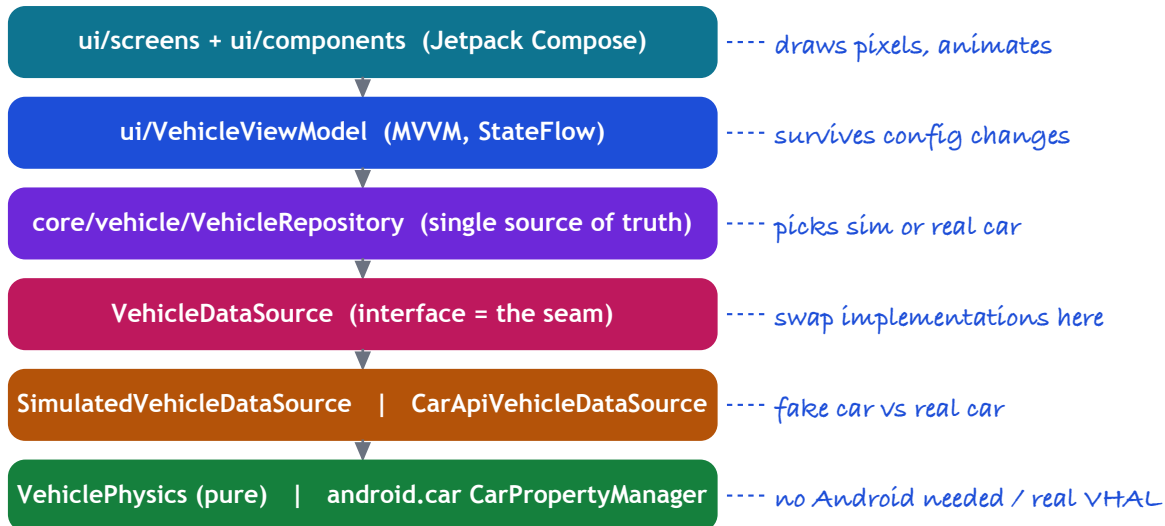
What's in this book

1 The big picture	3
Folder tree	4
2 Gradle & the manifest	5
3 core/vehicle: the heart	6
4 The other core packages	7
5 media/ and carapp/: plugging into the car's own UI	7
6 ui/: Compose, lessons and animations	8
7 Tests	9
8 Exercises: extend the app	9

1 The big picture

four rings, one direction of dependencies

The app is small (about 4,100 lines of Kotlin) but it is organised like a production IVI app. Dependencies point **inwards**: UI knows about core, core never knows about UI.



Data flows up as immutable `VehicleState`; commands flow down as method calls

BRAIN POWER

Why do you think the screens never import `android.car.*` directly? Think about (1) running on a phone, (2) unit testing, (3) a CarService crash.

Folder tree

```
SampleAutomotiveApp/
├── settings.gradle.kts      project name + repositories
├── build.gradle.kts        plugin versions (applied in :app)
├── gradle/libs.versions.toml version catalog: AGP, Kotlin, Compose, car-app...
├── app/
│   ├── build.gradle.kts    useLibrary("android.car"), minSdk 29, Compose
│   └── src/
│       ├── main/
│       │   ├── AndroidManifest.xml    features, car permissions, services
│       │   ├── res/xml/automotive_app_desc.xml    "media" + "template"
│       │   └── java/com/deepak/automotive/
│       │       ├── AutomotiveApp.kt    Application: creates VehicleRepository
│       │       ├── MainActivity.kt    runtime car permissions + Compose root
│       │       ├── core/
│       │       │   ├── car/
│       │       │   ├── vehicle/
│       │       │   ├── ux/
│       │       │   ├── can/
│       │       │   ├── security/
│       │       │   ├── power/
│       │       │   ├── ota/
│       │       │   └── audio/
│       │       ├── media/
│       │       ├── carapp/
│       │       └── ui/
│       │           ├── VehicleViewModel.kt
│       │           ├── theme/
│       │           ├── components/
│       │           ├── navigation/
│       │           └── screens/
│       └── test/
└── docs/
```

these PDFs + their Python sources

2 Gradle & the manifest

where the car-specific magic is declared

app/build.gradle.kts has one line you will not see in phone apps:

```
android {  
    compileSdk = 36  
    // compiles against platforms/android-36/optional/android.car.jar  
    useLibrary("android.car")  
}
```

That jar is a *stub*: at runtime the real classes are provided by the AAOS system image. On a phone they don't exist, which is why the manifest says `<uses-library android:name="android.car" android:required="false"/>` and why every Car API call is guarded by `CarAvailability.isAutomotive()`.

Manifest element	Why it is there
uses-feature android.hardware.type.automotive (required=false)	Marks the app as AAOS-capable while still installing on phones for learning.
uses-feature wifi / portrait / landscape required=false	Play Store filters AAOS apps that require hardware a head unit may lack.
uses-permission android.car.permission.*	CAR_SPEED & CAR_ENERGY are dangerous (runtime prompt); CAR_INFO/POWERTRAIN normal; CONTROL_CAR_CLIMATE is signature privileged (we request it to demonstrate the SecurityException).
meta-data com.google.android.gms.car.application → automotive_app_desc.xml	Declares we provide media and template experiences.
activity MainActivity + meta-data distractionOptimized=true	Allows the activity while driving; honoured for system apps. It is the learning playground.
service CarMediaBrowserService (MediaBrowserService action)	Lets the car's Media Center and Android Auto browse & control our content.
service ChargingStationCarAppService (CarAppService, category CHARGING)	Car App Library entry point rendered by the host as templates.

WATCH IT!

Forgetting `android:required="false"` on `uses-library android.car` makes the APK refuse to install on phones with `INSTALL_FAILED_MISSING_SHARED_LIBRARY`. On AAOS it always exists.

3 core/vehicle: the heart

a vehicleState goes in, a vehicleState comes out

File	Responsibility	Pattern
VehicleState.kt	Immutable snapshot: speed, rpm, gear, fuel, EV SoC, HVAC, signals, source, lastError	Value object / UDF state
VehicleDataSource.kt	Interface: state StateFlow + commands (setHvacTemperature, shift, setThrottle...)	Strategy / Port
SimulatedVehicleDataSource.kt	10 Hz coroutine loop that applies VehiclePhysics; safety interlocks	Adapter (fake)
VehiclePhysics.kt	Pure function step(state, throttle, brake, dt) → new state	Pure function, testable
CarApiVehicleDataSource.kt	Car.createCar → CarPropertyManager.registerCallback / setProperty, area lookup	Adapter (real)
VehicleRepository.kt	Owns both sources; flatMapLatest to expose the active one as a single StateFlow	Repository
VhalPropertyId.kt	Decodes 32-bit property IDs; catalog with change mode, access, permission	Domain model

How a speed change reaches the gauge (real car)



```
// CarApiVehicleDataSource.onPropertyChanged (simplified)
// VHAL speed is m/s, the UI wants km/h
VehiclePropertyIds.PERF_VEHICLE_SPEED -> s.copy(speedKmh = (v as Float) * 3.6f)
```

THERE ARE NO DUMB QUESTIONS

Q: Why StateFlow and not LiveData?

A: StateFlow is Kotlin-first, works in pure JVM tests, supports operators like flatMapLatest, and Compose collects it with collectAsStateWithLifecycle(). LiveData would tie core/ to Android.

Q: Why does CarApiVehicleDataSource re-subscribe inside the lifecycle listener?

A: Because CarService can crash and restart. createCar(..., WAIT_FOREVER, listener) calls you again with ready=true and **all old managers are dead**. Caching a manager across restarts is a classic AAOS bug.

Q: Why is speed multiplied by 3.6?

A: VHAL reports PERF_VEHICLE_SPEED in metres per second (SI units). Always check the unit in VehiclePropertyIds docs; an interviewer may ask exactly this.

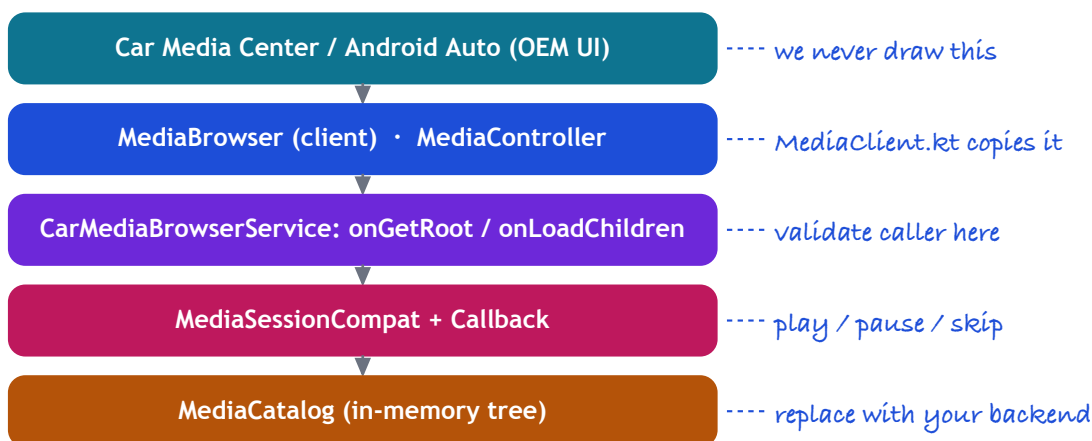
4 The other core packages

Package	What to read	Concept it teaches
core/ux	UxRestrictionsState + CarUxRestrictionsSource	Driver distraction; restrictions come from CarService
core/can	CanFrame, CanBus.arbitrate, encodeSpeed/decodeSpeed	Priority by ID, wired-AND bus, DBC scaling
core/security	SecOc (HMAC-SHA256 truncated MAC + freshness), SecureVault (AES-GCM in AndroidKeyStore)	Message authentication, replay protection, hardware-backed keys
core/power	PowerStateMachine.next(state, event)	CPMS, Garage Mode, suspend-to-RAM
core/ota	AbOtaUpdater.advance(state, version, bootOk)	A/B slots, verify, rollback
core/audio	FocusMatrix.interaction + AudioFocusRequester	Exclusive / concurrent (duck) / reject
core/car	CarAvailability.isAutomotive	FEATURE_AUTOMOTIVE check before touching android.car

INTERVIEW TIP

Notice that power, OTA, CAN and SecOC logic are **pure Kotlin functions** with unit tests. In interviews at Tier-1s, saying "I keep state machines pure and test them on the JVM, then wire them to Android at the edges" signals senior-level thinking.

5 media/ and carapp/: plugging into the car's own UI



MediaClient.kt connects to our own service exactly like the car does. The Media lesson screen is therefore a working *test harness* for the service.

ChargingStationCarAppService.kt: CarAppService → Session → StationListScreen (ListTemplate) → StationDetailScreen (PaneTemplate) with a Navigate action that fires `CarContext.ACTION_NAVIGATE` with a geo: URI.

WATCH IT!

HostValidator.ALLOW_ALL_HOSTS_VALIDATOR is used **only in debug builds**. In release the service restricts hosts to an allowlist of package names + signing certificates. Shipping ALLOW_ALL is a real security finding.

6 ui/: Compose, lessons and animations

File	Role
navigation/Lessons.kt	Registry of 15 lessons: route, title, icon, colour, Head First notes
navigation/AppNavHost.kt	NavHost with slide/fade transitions; wraps every lesson in LessonScaffold
components/LessonScaffold.kt	Header + collapsible notes panel (AnimatedVisibility) + scrollable demo
components/Gauge.kt	Canvas gauge; needle uses spring() so it sweeps like a stepper motor
components/CarButton.kt	76dp-friendly buttons with animated selection colour
screens/*	One file per lesson; each observes VehicleViewModel.state

Animation toolbox used (learn these names for Compose interviews)

API	Used in	Why
animateFloatAsState + spring	Gauge, fuel bar, temperature	Smooth value changes between samples
rememberInfiniteTransition	Home road, turn signals, equalizer, ultrasonic waves	Looping ambient motion
AnimatedContent	Gear letter, track title, OTA phase, Car App screens	Enter/exit when content changes
AnimatedVisibility	Notes panel, video lock overlay, errors	Show/hide with expand/fade/scale
Animatable	Home tile stagger, CAN frame travel, VHAL row flash	Imperative, coroutine-driven
withInfiniteAnimationFrameMillis	HVAC fan	Frame-accurate physics (rotation $\propto$ fan speed)
animateColorAsState / lerp	Zones, slots, restrictions	Colour communicates state
animateContentSize	UX restricted list/text	Layout grows/shrinks smoothly

7 Tests

Test class	What it proves
CanBusTest	Lowest ID wins; speed signal round-trips; >8 bytes rejected
SecOcTest	Fresh authentic frame accepted; replay, forged MAC and tampered payload rejected
StateMachinesTest	Garage Mode → suspend → fast resume; invalid event ignored; OTA success & rollback
VehicleTest	VHAL ID decoding; no motion in Park; acceleration in Drive; UX restrictions; focus matrix

INTERVIEW TIP

Next level (mention it!): instrumentation tests on an AAOS emulator using `adb shell cmd car_service inject-vhal-event 0x11600207 20` to fake speed, Compose UI tests for restricted states, and CTS/ CTS-Verifier for OEM builds.

8 Exercises: extend the app

the best way to own a codebase is to change it

SHARPEN YOUR PENCIL

- **Add a property:** add TIRE_PRESSURE (area WHEEL) to VhalCatalog and show it in the VHAL screen. Hint: decode its ID first; area type should be WHEEL.
- **Seat heating:** add HVAC_SEAT_TEMPERATURE to VehicleDataSource with levels -3..3 and a heated-seat animation.
- **Night mode:** switch AutomotiveTheme to a dimmer palette when state.nightMode is true.
- **New restriction:** hide the Media browse tree beyond depth 3 when LIMIT_CONTENT is active.
- **Car App Library:** add a MapWithContentTemplate screen (requires car API level 7).
- **CAN FD:** allow 64-byte frames when a flag is set and adjust the validation + tests.

Answers / hints

- TIRE_PRESSURE is 0x17600309: group SYSTEM (1), area WHEEL (7), type FLOAT (0x60), id 0x0309.
- Seat temperature is an INT32 SEAT-area property; negative values mean cooling.
- Collect nightMode in AutomotiveTheme and swap colour schemes; AAOS also has UiModeManager night mode.
- Use ux.maxContentDepth from UxRestrictionsState.

BULLET POINTS

- core/ = pure logic and Android adapters; ui/ = Compose only.
- VehicleDataSource is the seam between simulator and real VHAL.
- useLibrary("android.car") + uses-library required=false + runtime feature check.
- Media and templates: the car draws the UI; we provide data and callbacks.
- State machines are pure and unit tested.